

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1408

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS: 50

Annexure A

**Duration :60 Mins
On Class
Total Marks:50**

SCENARIO BASED TASK

Code of Conduct:

I Ranjitha R (192421285) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

SCENARIO BASED TASK

Q1. A compiler development team is implementing a simple desk-calculator module that evaluates arithmetic expressions entered by users. The calculator supports addition, subtraction, multiplication, division, parentheses, and single-digit operands. The compiler uses a Syntax-Directed Definition (SDD) to compute the value of each expression during parsing.

The grammar used by the calculator is:

$$S \rightarrow E \text{ n}$$
$$E \rightarrow E + T \\ | E - T \\ | T$$
$$T \rightarrow T * F \\ | T / F \\ | F$$
$$F \rightarrow (E) \\ | \text{digit}$$

For the input expression:

$$(6 + 4) * (4 + 3)$$

Analyze the grammar and design an appropriate SDD for evaluating arithmetic expressions. Construct the annotated parse tree for the given input and show the attribute values computed at each relevant node. Finally, determine the final value produced by the calculator.

Q2. A compiler development team is testing the syntax-directed translation module of an arithmetic expression processor. During testing, the expression

$$a * b - c + 8$$

is given as input. The compiler must correctly represent the expression according to operator precedence and associativity and use a Syntax-Directed Definition (SDD) to evaluate the expression.

Analyze the given expression, construct its syntax tree considering the precedence and associativity of the operators, and formulate an appropriate SDD for evaluating the expression. Show the attribute evaluation sequence for the given expression.

Q3. A compiler development team is designing a semantic-analysis module that uses inherited and synthesized attributes to pass information between grammar symbols during translation.

The team needs to verify whether the semantic rules used in the grammar satisfy the conditions of an L-attributed definition, so that the attributes can be evaluated from left to right during parsing.

Consider the production rules:

$$\begin{aligned} A &\rightarrow P Q \\ A &\rightarrow X Y \end{aligned}$$

Each of the non-terminals A, P, Q, X, and Y has two attributes:

- s – synthesized attribute
- i – inherited attribute

The compiler team proposes the following semantic rules:

Rule 1:

$$\begin{aligned} P.i &= A.i + 2 \\ Q.i &= P.i + A.i \\ A.s &= P.s + Q.s \end{aligned}$$

Rule 2:

$$\begin{aligned} X.i &= A.i + Y.s \\ Y.i &= X.s + A.i \end{aligned}$$

Analyze the two sets of semantic rules with respect to the conditions of an L-attributed definition. Determine which rule satisfies the requirements of an L-attributed definition and which rule violates them. Justify your answer by examining the dependency of inherited and synthesized attributes and their left-to-right evaluation order.

Q4. A compiler development team is implementing an expression translator for a calculator application. The translator must evaluate arithmetic expressions and generate an appropriate intermediate code representation using an S-attributed Syntax-Directed Definition (SDD). The syntax analyzer uses LR parsing to process the input from left to right and construct the required parse structure.

The compiler uses the following grammar:

$$\begin{aligned} S &\rightarrow E N \\ E &\rightarrow E + T \\ E &\rightarrow E - T \\ E &\rightarrow T \\ T &\rightarrow T * F \\ T &\rightarrow T / F \\ T &\rightarrow F \\ F &\rightarrow (E) \\ F &\rightarrow \text{digit} \\ N &\rightarrow ; \end{aligned}$$

The input expression provided to the translator is:

$2 + 5 * 6 ;$

Analyze the given grammar and design an S-attributed Syntax-Directed Definition for generating the corresponding code fragment (translator). Then trace the LR parsing process for the input string $2 + 5 * 6 ;$ using a parser stack, showing the sequence of shift and reduce operations. Finally, show the generated translation for the given input.

Q5.A compiler development team is implementing a top-down expression translator for an arithmetic calculator. The translator must process arithmetic expressions containing addition, multiplication, parentheses, and single-digit operands. During translation, the compiler should use a Syntax-Directed Definition (SDD) to compute the value of the expression and annotate the corresponding parse tree.

The grammar used by the translator is:

$S \rightarrow E n$

$E \rightarrow E + T$
 $\quad | T$

$T \rightarrow T * F$
 $\quad | F$

$F \rightarrow (E)$
 $\quad | \text{digit}$

The compiler is required to process the following input:

$3 * 4 + 8 n$

Analyze the given grammar and formulate a suitable SDD for top-down translation. Construct the annotated parse tree for the given input expression, showing the attribute values at the appropriate nodes. Finally, trace the top-down evaluation and determine the final value generated by the translator.

RUBRICS FOR CALCULATION

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q1. SDD for Arithmetic Expression Evaluation:

Given:-

$$(6+4) * (4+3)$$

i) SDD

Production

$$S \rightarrow E_n$$

$$E \rightarrow E_1 + T$$

$$E \rightarrow E_1 - T$$

$$E \rightarrow T$$

$$T \rightarrow T_1 * F$$

$$T \rightarrow T_1 / F$$

$$T \rightarrow F$$

$$F \rightarrow (E)$$

$$F \rightarrow \text{digit}$$

Semantic Rule

$$S.val = E.val$$

$$E.val = E_1.val + T.val$$

$$E.val = E_1.val - T.val$$

$$E.val = T.val$$

$$T.val = T_1.val * F.val$$

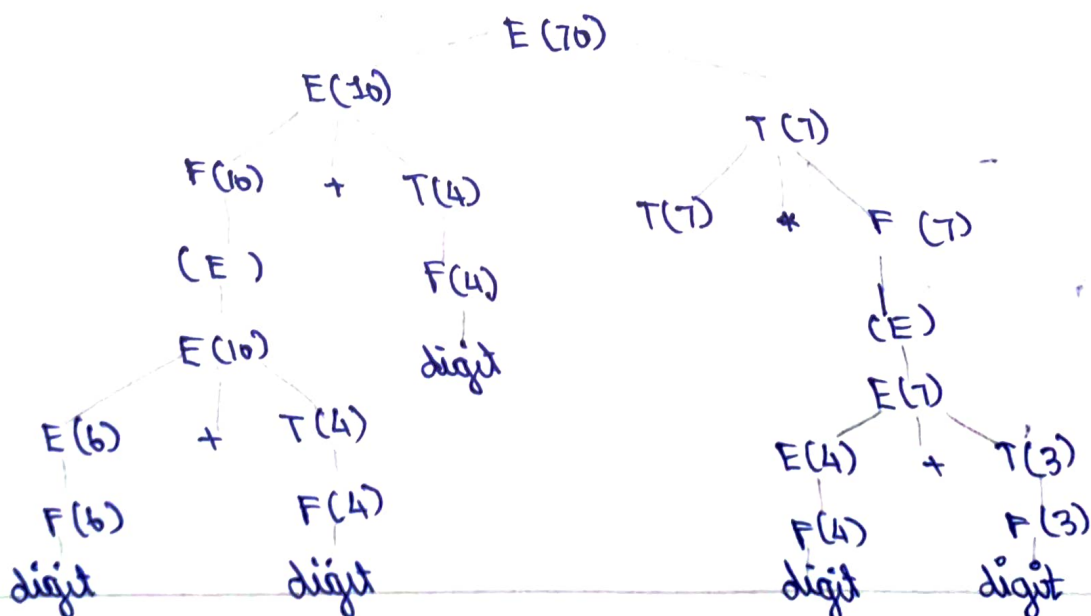
$$T.val = T_1.val / F.val$$

$$T.val = F.val$$

$$F.val = E.val$$

$$F.val = \text{digit.lexval}$$

ii) Annotated Parse Tree



3. Attribute Evaluation:-

$$6 + 4 = 10 \rightarrow F.val = 10$$

$$4 + 3 = 7 \rightarrow F.val = 7$$

$$10 \times 7 = 70 \rightarrow E.val = 70$$

Q2. SDD for $a * b - c + 8$

i) operation precedence:-

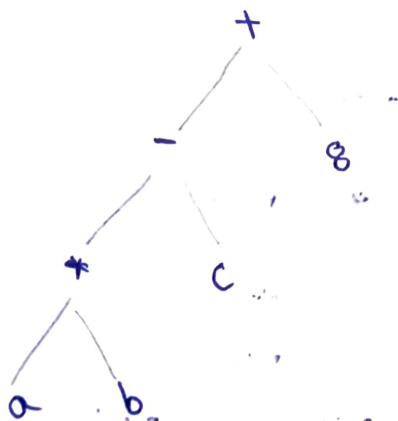
The expression is : $a * b - c + 8$

multiple has higher precedence than subtraction and addition.

Addition and subtraction have the same precedence and are evaluated from left to right.

Therefore, $(a * b) - c + 8$

ii) Syntax Tree.



iii) SDD

Production

Semantic Rule

$$E \rightarrow E_1 + T$$

$$E.val = E_1.val + T.val$$

$$E \rightarrow E_1 - T$$

$$E.val = E_1.val - T.val$$

$$E \rightarrow T$$

$$E.val = T.val$$

$$T \rightarrow T_1 * F$$

$$T.val = T_1.val * F.val$$

$$T \rightarrow F$$

$$T.val \rightarrow F.val$$

$$F \rightarrow id$$

$$F.val = id.val$$

$$F \rightarrow digit$$

$$F.val = digit.lexval$$

iv) Attribute Evaluation Sequence.

$$a * b \rightarrow T.val = a * b$$

$$(a * b) - c \rightarrow E.val = (a * b) - c$$

$$((a * b) - c) - 8 \rightarrow E.val = ((a * b) - c) - 8$$

Q3. L-Attributed Definition

Given:-

$$A \rightarrow PQ$$

$$A \rightarrow xy$$

s $\rightarrow$ synthesized attribute

i $\rightarrow$ inherited attribute

Rule 1:

$$P.i = A.i + 2$$

$$Q.i = P.i + A.i$$

$$A.s = P.s + Q.s$$

Analysis.

For $P.i$

$P.i$ depends only on the inherited attribute of the Parent A .

For $Q.i$

Q is to the right P

Therefore, $Q.i$ can depend on an attribute of the symbol on its left (P) and the parent (A)

For $A.s$

A synthesized attribute can depend on the attributes of the children.

Rule 1 satisfies the L-attributed definition

Rule 2.

$$X.i = A.i + Y.s$$

$$Y.i = X.s + A.i$$

For $X.i$

Y is to the right X

So the inherited attribute of X depends on the

Synthesized attribute of symbol to its right

For γ :

x is to the left of γ

however, the first dependency is already invalid

Rule 2 does not satisfy the L-attributed definition

Q4. S-Attributed SDD and LR parsing

Given:

$2 + 5 * 6;$

i) SDD

Production

$E \rightarrow E_1 + T$

$E \rightarrow E_1 - T$

$E \rightarrow T$

$T \rightarrow T_1 * F$

$T \rightarrow T_1 / F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

$S \rightarrow EN$

$N \rightarrow ;$

Semantic Rule:

$E.val = E_1.val + T.val$

$E.val = E_1.val - T.val$

$E.val = T.val$

$T.val = T_1.val * F.val$

$T.val = T_1.val / F.val$

$T.val = F.val$

$F.val = E.val$

$F.val = \text{digit} \cdot \text{lex val}$

$S.val = E.val$

$N.val = 0$

ii) operation precedence:-

$$2 + (5 * 6)$$

$$5 * 6 = 30$$

$$2 + 30 = 32$$

iii) LR parsing Trace

Step	Stack	Input	Action
1	\$	2 + 5 * 6 ; \$	shift 2
2	\$ 2	+ 5 * 6 ; \$	Reduce $F \rightarrow \text{digit}$
3	\$ F	+ 5 * 6 ; \$	Reduce $T \rightarrow F$
4	\$ T	+ 5 * 6 ; \$	Reduce $E \rightarrow T$
5	\$ E	+ 5 * 6 ; \$	shift +
6	\$ E +	5 * 6 ; \$	shift 5
7	\$ E + 5	* 6 ; \$	Reduce $F \rightarrow \text{digit}$
8	\$ E + F	* 6 ; \$	Reduce $T \rightarrow F$
9	\$ E + T	* 6 ; \$	shift *
10	\$ E + T *	6 ; \$	shift 6
11	\$ E + T * 6	; \$	Reduce $F \rightarrow \text{digit}$
12	\$ E + T * F	; \$	Reduce $T \rightarrow F * T$
13	\$ E + T	; \$	Reduce $E \rightarrow E + T$
14	\$ E	; \$	shift ;

15	$\$E;$	$\$$	Reduce $N \rightarrow ;$
16	$\$EN$	$\$$	Reduce $S \rightarrow EN$
17	$\$S$	$\$$	Accept.

Q5. Top-Down Translation using RDR

Given:

$$3^x A + 9n$$

i) Grammar:-

$$S \rightarrow EN$$

$$E \rightarrow E + T$$

$$E \rightarrow T$$

$$T \rightarrow T * F$$

$$T \rightarrow F$$

$$F \rightarrow (E)$$

$$F \rightarrow \text{digit}$$

ii) RDR

Production

$$S \rightarrow EN$$

$$E \rightarrow E_1 + T$$

$$E \rightarrow T$$

$$T \rightarrow T_1 * F$$

$$T \rightarrow F$$

Semantic Rule

$$S.val = E.val$$

$$E.val = E_1.val + T.val$$

$$E.val = T.val$$

$$T.val = T_1.val * F.val$$

$$T.val = F.val$$

$$F \rightarrow (E)$$

$$F.Val = E.Val$$

$$F \rightarrow \text{digit}$$

$$F.Val = \text{digit} \cdot \text{lexVal}$$

iii) Expression Evaluation :-

$$3 * 4 + 8$$

$$(3 * 4) + 8$$

$$3 * 4 = 12$$

$$12 + 8 = 20$$

iv) Annotated Parse Tree

